

Data Structure and Algorithm

COURSE CODE: CS-201

TOPIC: STACK AND ITS OPERATIONS



Goal for today

To introduce :

- Stacks
- Algorithms for insertion and deletion in stacks
- Applications of stacks

CLO Covered

CLO 2: **Apply** and design various data structures methods for specified applications. GA4

Stacks

There are certain frequent situations in computer science when one wants to restrict insertion and deletions so that they can take place only at the beginning or at the end not in the middle.

- Stack (LIFO)
- Queue (FIFO)

These are actually conditions on linked list and linear arrays.

Stack

“A *Stack* is a special kind of list in which all insertions and deletions take place at one end, called the *Top*”

Other Names

- Pushdown List
- Last In First Out (LIFO)

Stack

- It is called as stack because it behaves like a real-world stack, piles of books, etc.
- A Stack is an abstract data type with a pre-defined capacity, which means that it can store the elements of a limited size.
- Stores a set of elements in a particular order
- It is a data structure that follows some order to insert and delete the elements, and that order can be
 - LIFO (Last – In – First – Out)
 - FILO (First – In – Last – Out)

Stack

Stack principle: **LAST IN FIRST OUT**= **LIFO**

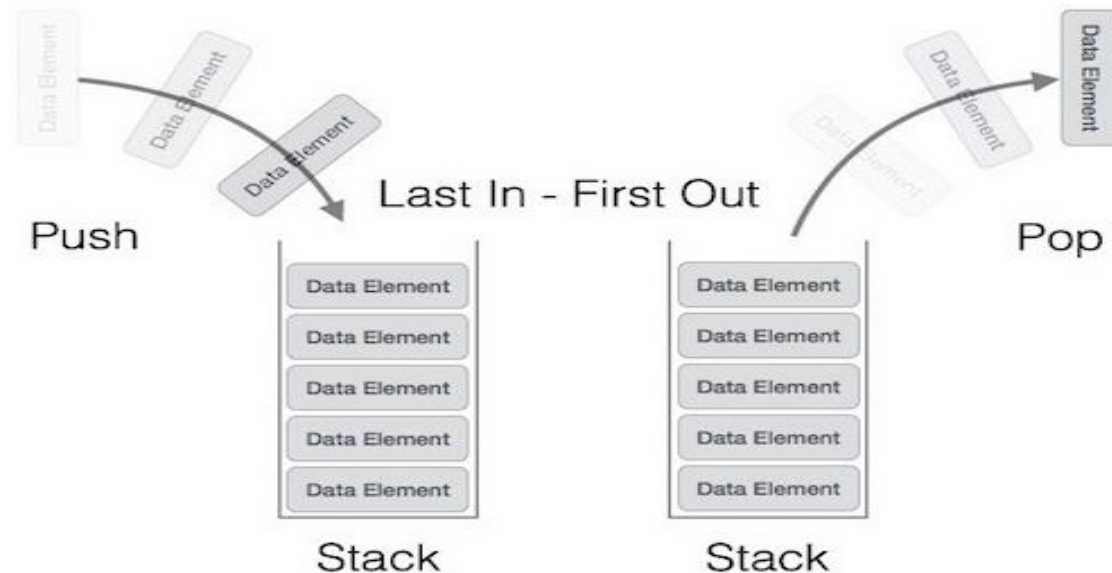
It means: the last element inserted is the first one to be removed

Which is the first element to pick up?

Stack

Two basic operations are associated with Stack

- Push : Insert an element into a stack
- Pop : Delete an element from a stack



Real time applications of stacks

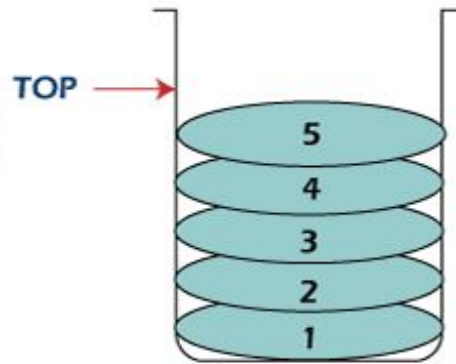
Backtracking features:

- Undo feature in word.
- To previous choice point in a game.
- To retrieve recently used objects in cache.

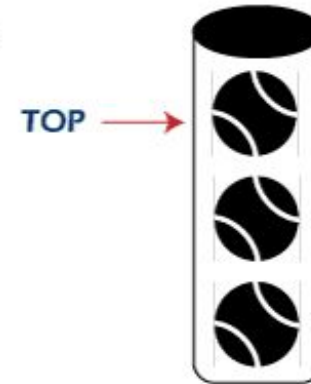
Stack (Examples)



Stack of Coins



Stack of Plates



Can of Tennis Balls



Stack of Books

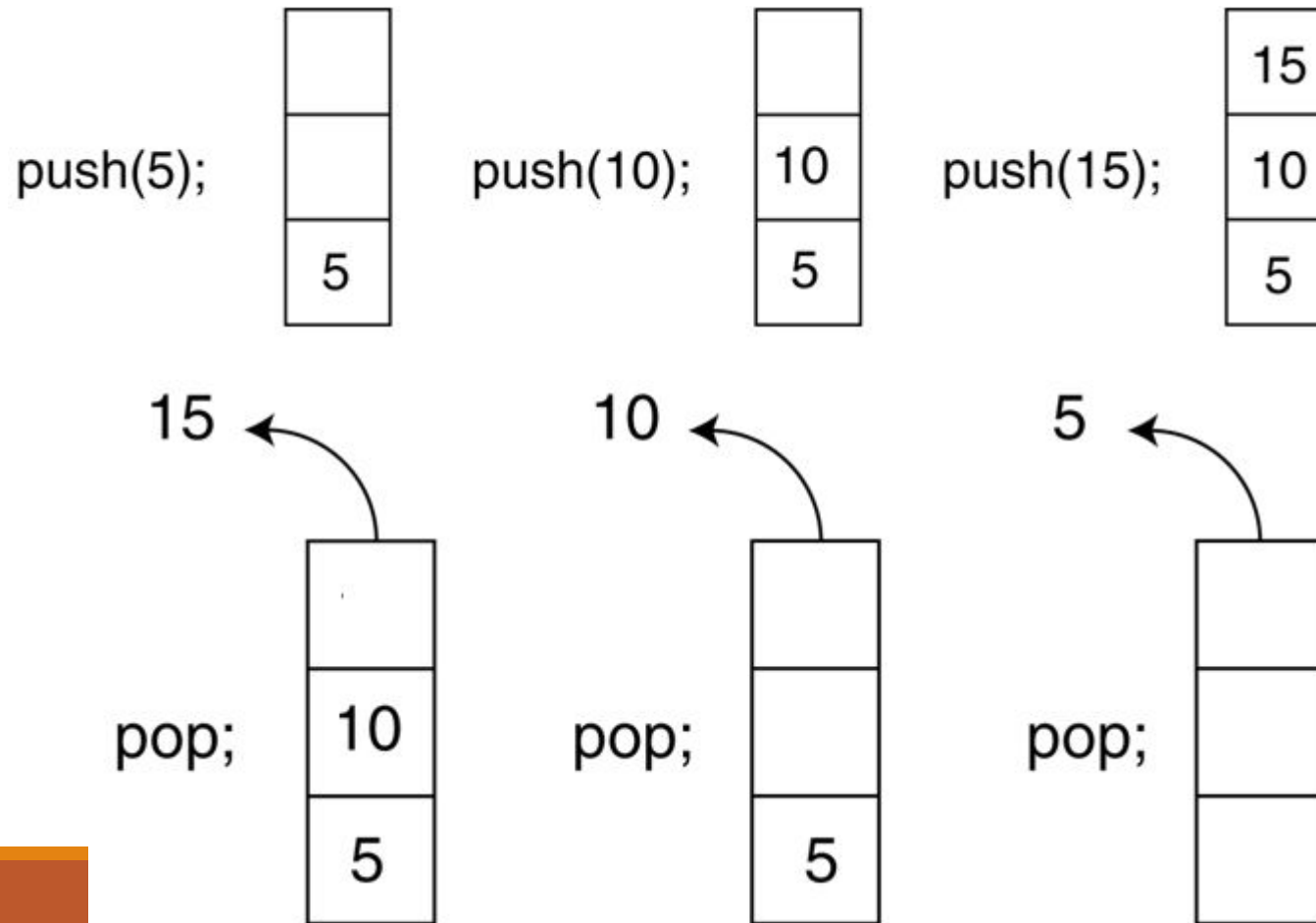
← → ↻ ⓘ www.log2base2.com/2017/06/blog-post.html

Browser back button implemented using stack.
Last In First Out

Common Operations on Stacks

- **push():** When we insert an element in a stack then the operation is known as a push. If the stack is full then the overflow condition occurs.
- **pop():** When we delete an element from the stack, the operation is known as a pop. If the stack is empty means that no element exists in the stack, this state is known as an underflow state.
- **isEmpty():** It determines whether the stack is empty or not.
- **isFull():** It determines whether the stack is full or not.'
- **peek():** It returns the element at the given position.
- **display():** It prints all the elements available in the stack.

Push and Pop Operations on Stack



Stacks Representation

Stack data structure can be implemented in two ways. They are as follows...

- Linear Array (Static memory allocation)
- One way Linked List (Dynamic memory allocation)

Static and Dynamic Stacks

Stack using array

- When a stack is implemented using an array, that stack can organize an only limited number of elements.
- **Static type**, i.e. they have a **fixed size**, and are *implemented as arrays*.

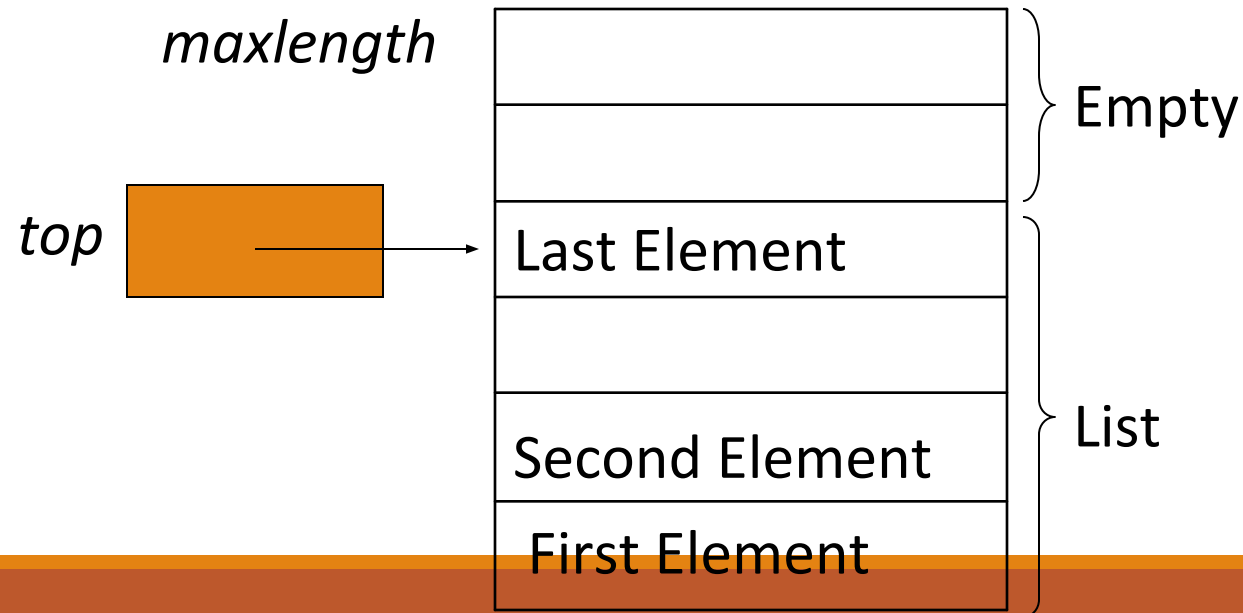
Stack using linked list

- When a stack is implemented using a linked list, that stack can organize an unlimited number of elements.
- **Dynamic type**, i.e. they **grow in size** as needed, and *implemented as linked lists*

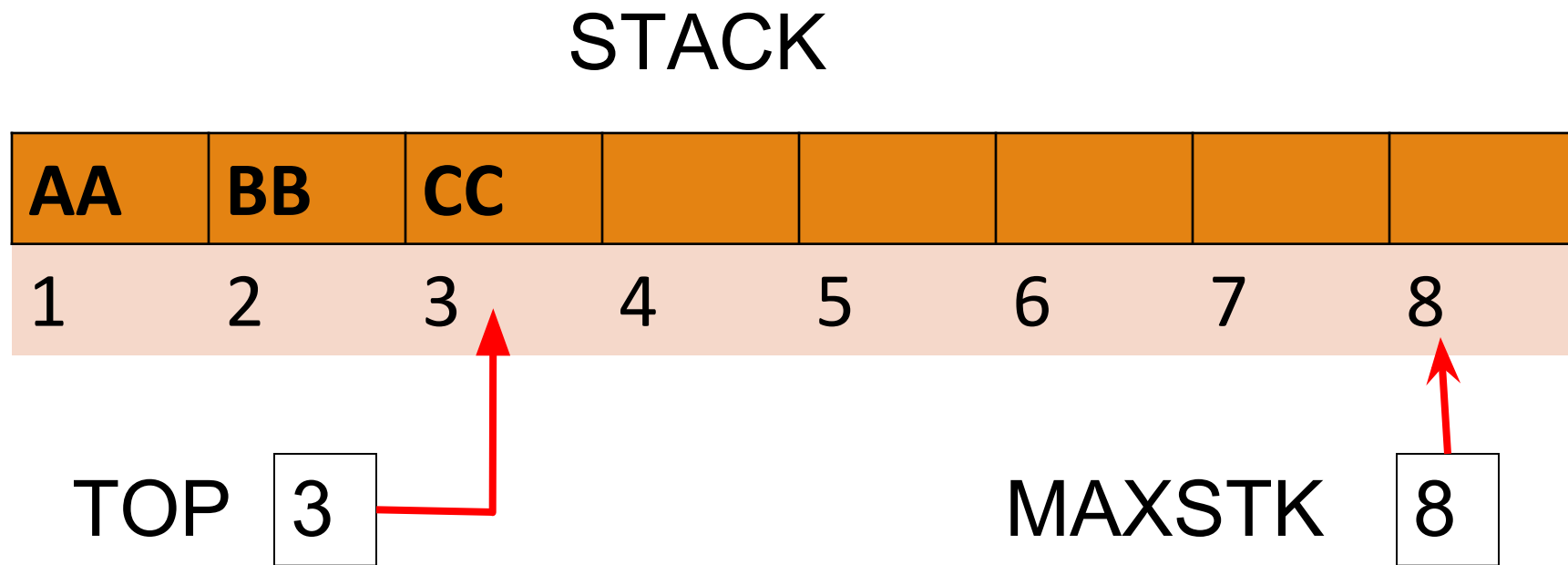
An Array Implementation of Stacks

First Implementation

- Elements are stored in contiguous cells of an array.
- New elements can be inserted to the top of the list.



Array Representation of Stack



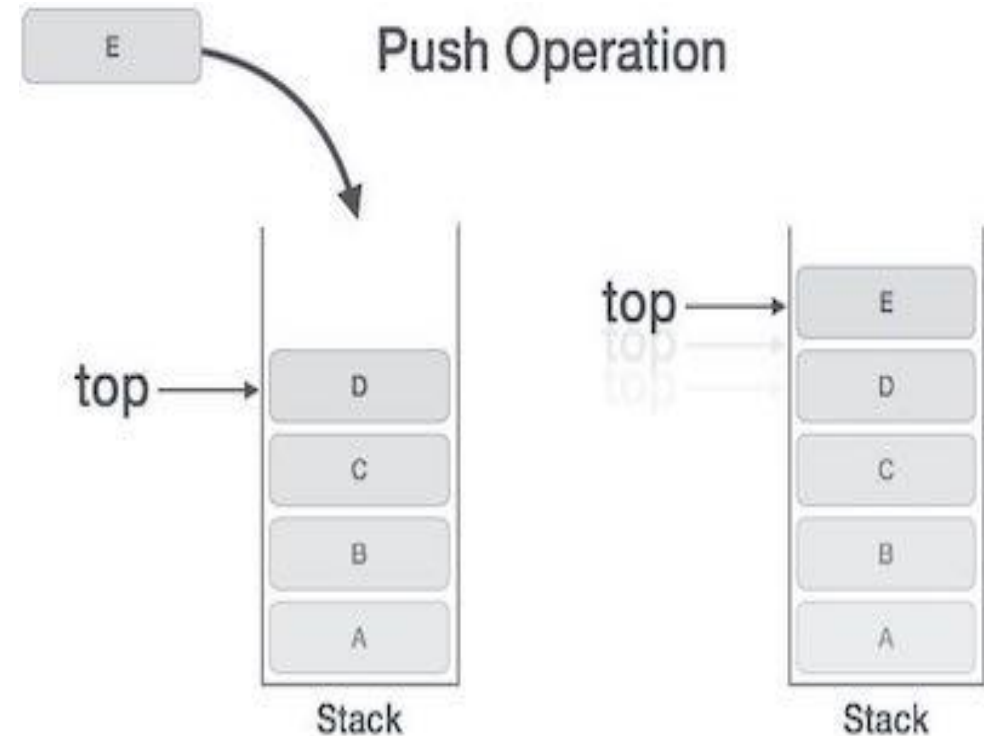
TOP = 0 or TOP = NULL will indicate that the stack is empty

An Array Implementation of Stacks

- A stack data structure can be implemented using a one-dimensional array.
- But stack implemented using array stores only a fixed number of data values.
- This implementation is very simple. Just define a one dimensional array of specific size and insert or delete the values into that array by using LIFO principle with the help of a variable called 'top'.
- Initially, the top is set to -1.
- Whenever we want to insert a value into the stack, increment the top value by one and then insert.
- Whenever we want to delete a value from the stack, then delete the top value and decrement the top value by one.

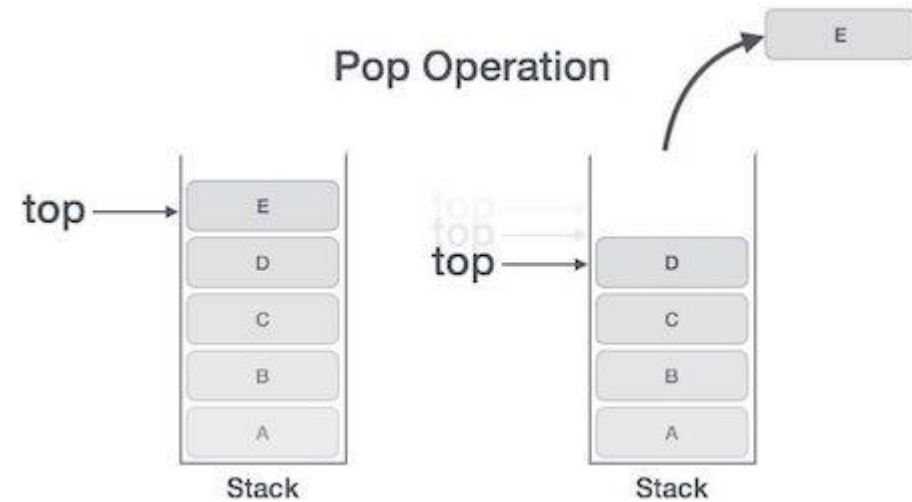
Push(): Insertion in stack

```
if(top == MAXSIZE)
    print: Overflow, Exit [ Stack already filled]
else
    top = top + 1;
    stack[top] = data;
```



Pop():Deletion in stack

```
if(top == -1)
    print: Underflow, Exit [ Stack is empty]
else
    data = stack[top];
    top = top - 1;
```



```

include <conio.h>

#include <iostream.h>

#include <process.h>

#define STACKSIZE 10 // int const STACKSIZE = 10;

// global variable and array declaration

int Top=-1;

int Stack[STACKSIZE];

void Push(int); // functions prototyping

int Pop(void);

bool IsEmpty(void);

bool IsFull(void);

void Traverse(void);

int main( )

{ int item, choice;

    while( 1 )

    {

cout<< "\n\n\n\n\n";

cout<< " ***** STACK OPERATIONS ***** \n\n";

cout<< " 1- Push item \n 2- Pop Item \n";

cout<< " 3- Traverse / Display Stack Items \n 4- Exit.";

cout<< " \n\n\t Your choice ---> ";

cin>> choice;

```

```

switch(choice)

{ case 1: if(IsFull())cout<< "\n Stack
Full/Overflow\n";

    else

    { cout<< "\n Enter a number: "; cin>>item;

    Push(item); }

    break;

case 2: if(IsEmpty())cout<< "\n Stack is empty) \n";

    else

    {item=Pop();

cout<< "\n deleted from Stack = "<<item<<endl;}

    break;

    case 3: if(IsEmpty())cout<< "\n Stack is empty)
\n";

    else

    { cout<< "\n List of Item pushed on Stack:\n";

    Traverse();

    }

    break;

```

```
case 4: exit(0);
```

```
default:
```

```
cout<< "\n\n\t Invalid Choice: \n";
```

```
} // end of switch block
```

```
    } // end of while loop
```

```
} // end of of main() function
```

```
void Push(int item)
```

```
{
```

```
    Stack[++Top] = item;
```

```
}
```

```
int Pop( )
```

```
{
```

```
    return Stack[Top--];
```

```
}
```

```
bool IsEmpty( )
```

```
{ if(Top == -1 ) return true else return false; }
```

```
bool IsFull( )
```

```
{ if(Top == STACKSIZE-1 ) return true else return false; }
```

```
void Traverse( )
```

```
{ int TopTemp = Top;
```

```
    do{ cout<< Stack[TopTemp--]<<endl;} while(TopTemp>= 0);
```

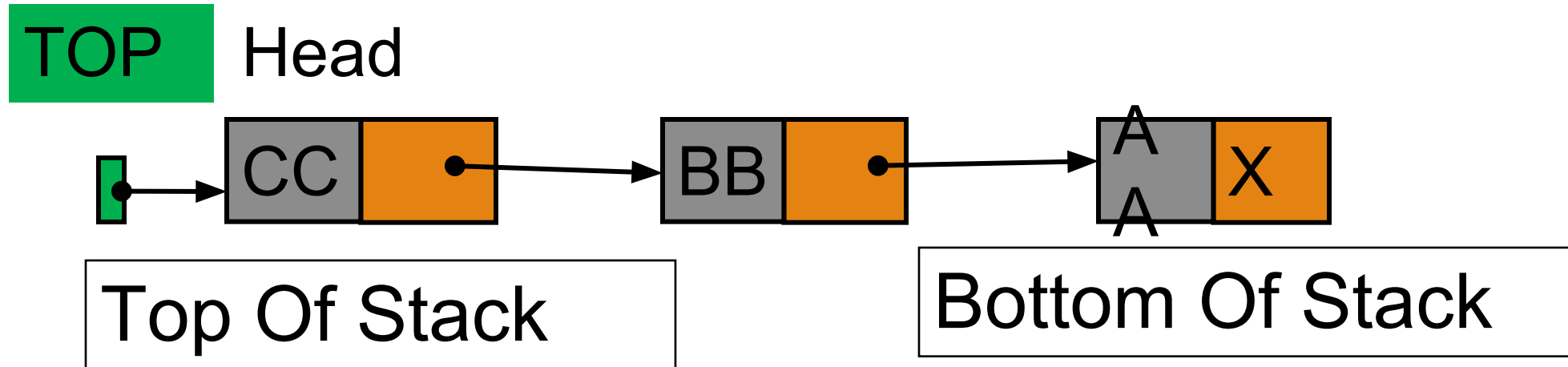
```
}
```

Stacks algorithms using linked list

- Insertion

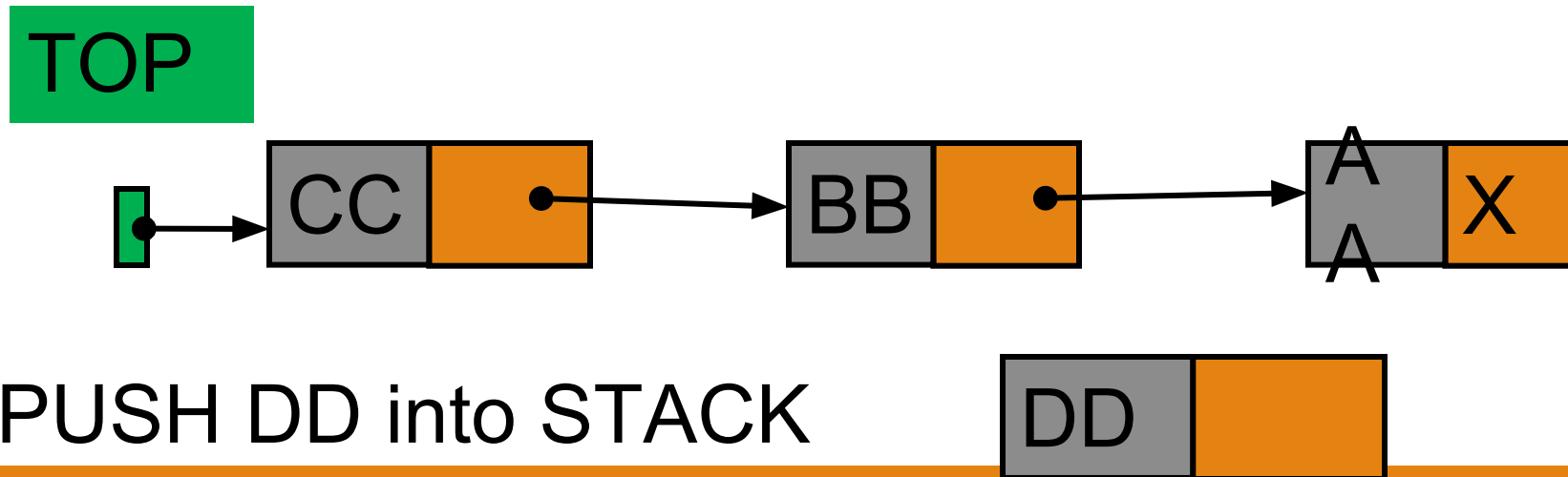
- Deletion

Linked List Representation of Stack



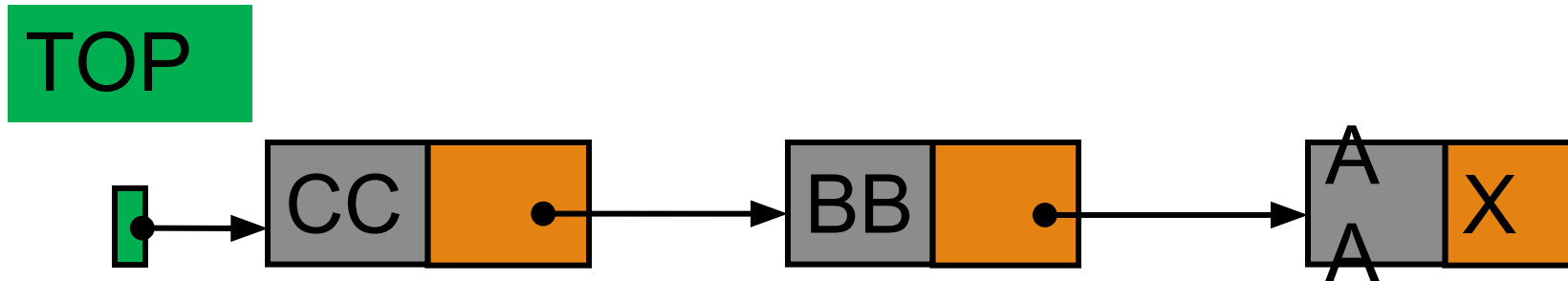
PUSH Operation

Push operation into the stack is accomplished by inserting a node into the front of the list [**Insert it as the first node in the list**]

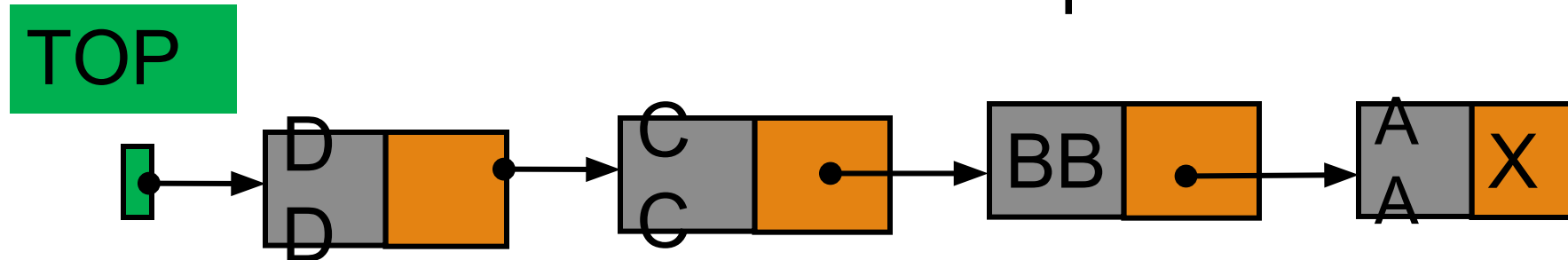


PUSH Operation

STACK before PUSH Operation



STACK After PUSH Operation



PUSH Operation using linked list

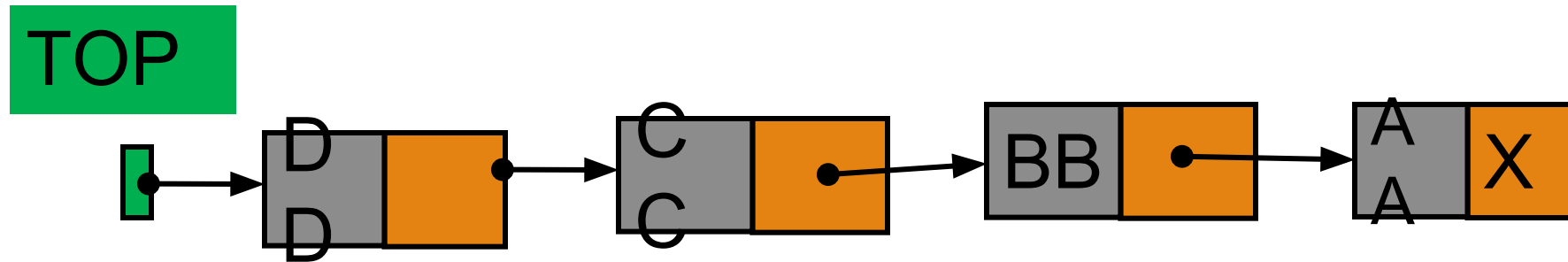
```
struct node *newnode;  
newnode=(struct node*)malloc(sizeof(struct node));  
newnode ->data=item; //any type of data such as number or name  
newnode ->link=top;  
top=newnode;
```

POP Operation

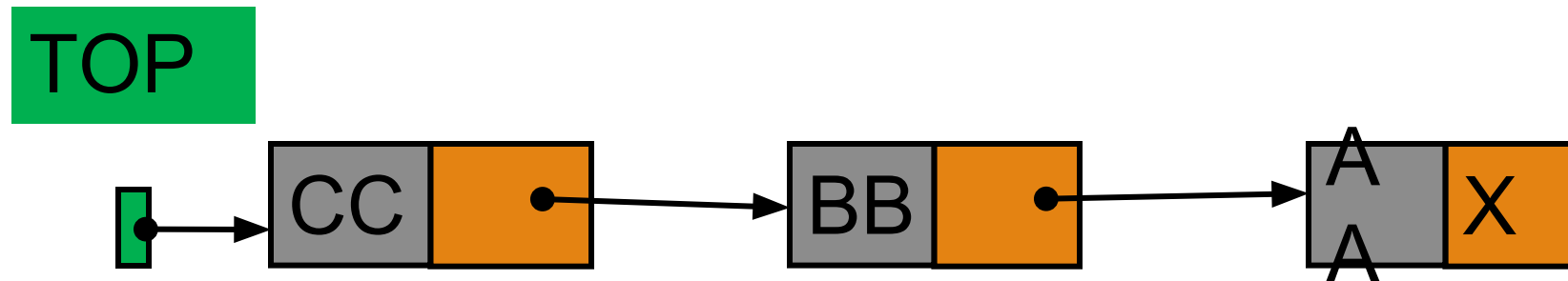
POP operation is accomplished by deleting the node pointed to by the TOP pointer [**Delete the first node in the list**]

POP Operation

STACK before POP Operation



STACK After POP Operation



POP Operation using linked list

```
deletion from top
struct node * temp;
temp=top;
if top == null
    then write underflow and exit
print: top ->data;
top=top->link;
free( temp);
```

Application of stacks

- Reversal of strings.
- Checking validity of brackets.
- Converting infix to postfix and prefix.
- Evaluating the postfix expression.

Your Turn

Consider the following stack of characters, where STACK is allocated N=8 memory cells:

- STACK: A, C, D, F, K, __, __, __,

“__” is used to represent empty memory cells. Describe the stack as the following operations take place:

- | | |
|---------------------|---------------------|
| a. POP(STACK, ITEM) | e. POP(STACK, ITEM) |
| b. POP(STACK, ITEM) | f. PUSH(STACK, R) |
| c. PUSH(STACK, L) | g. PUSH(STACK, S) |
| d. PUSH(STACK, P) | h. POP(STACK, ITEM) |